

平时作业一

2312966 林晖鹏 计算机学院

一、浮点数有没有移位操作和扩展操作？为什么？

我觉得这个问题得分两层看。

移位操作 像整数那样直接用 `>>` `<<` 对浮点数整体移位，在 C 里肯定是不行的，编译直接报错。原因是整数的二进制每一位都是数值的一部分，左移右移就相当于乘 2 除 2。但浮点数不是这样，它内部是“符号位+指数位+尾数位”分开存的，如果硬把整个 32 位一起移，符号、指数、尾数全都混在一起了，移出来的结果根本没意义。

不过 CPU 自己做浮点加减法的时候，内部会有一个“对阶”的步骤（比如： $1.25 \times 2^3 + 1.00 \times 2^1$ ），就是把指数小的数的尾数右移，让两个数的小数点对齐，这个其实有点像移位，只是它对程序员来说是透明的，我们看不到也操作不了。

扩展操作 这个是有。最典型的就是把 `float` 转成 `double`，比如：

```
float a = 1.5f;
double b = a;
```

这中间就会把 32 位的单精度格式扩展成 64 位的双精度格式。但要注意的是，如果最开始存成 `float` 的时候已经舍入过一次，转成 `double` 也只是忠实地保存那个近似值，并不能把已经丢掉的精度找回来。

二、为什么整数除 0 会发生异常，而浮点数除 0 不会？

主要原因是整数和浮点数的表示能力不一样。

整数能表示的范围是有限的、离散的，比如 -2, -1, 0, 1, 2.....这套集合里根本就没有“无穷大”这种东西。所以如果让整数除以 0，硬件发现自己给不出一个合法的整数结果，就直接报异常了。

浮点数用的是 IEEE 754 标准，它事先定义好了几个特殊值，比如 $+\infty$ 、 $-\infty$ 和 `NaN`。所以当我们写 `1.0 / 0.0` 时，结果可以直接用 `+inf` 来表示，`0.0 / 0.0` 则用 `NaN` 表示，程序不会崩。我以前在代码里故意试过，Python 里直接打印出 `inf`，程序照常运

行。所以可以说，浮点数除 0 不异常是因为它内置了“异常值”的表示，而不是真的没发生错误。

（当然，如果特意设置浮点异常捕获，浮点除 0 也可能触发信号，但那不是默认情况，这里就不展开说明了。）

三、为什么同一个实数赋值给 float 型变量和 double 型变量，输出结果会有所不同？

因为 float 和 double 的精度根本不一样。float 只有 32 位，有效数字大概 6-7 位十进制；double 64 位，有效数字大概 15-16 位。所以同样一个实数，比如 3.141592653589793，存进 float 时只能截断或舍入到它所能表示的最接近的值，存进 double 则可以保留更多位数。

另外还有一个更底层的原因：很多我们看着很短的十进制小数，比如 0.1、0.2，在二进制下其实是无限循环小数。计算机存它们的时候，不管是 float 还是 double，都不可能存完全精确，只能存一个近似值。只是 float 截得比较短，double 截得比较长，最后打印出来自然就不一样了。

四、为什么 float 情况下输出的结果会比原来的大？这是根本原因还是随机发生的？

这不是随机的，是舍入造成的必然结果。

因为 float 不能精确表示原来的实数，它只能在两个最近的 float 可表示值之间选一个。如果离上边那个值更近，结果就会比原来的数大一点；如果离下边的更近，就会小一点。比如 0.1 转成 float，实际存进去的值大约是 0.10000000149，就比 0.1 大了那么一点点。换另外一个数，比如 0.3，存成 float 后可能是 0.29999999..... 就又变小了。

所以“输出比原来大”只是某种具体舍入方向的表现，不是每次都会大，更不是随机乱变。但是，在同一个平台、同一个编译器下，同一个十进制数转成 float 的结果是固定的，你跑多少遍都一样。
